TALLER II

CORTE I

DOCENTE

ELIECER MONTERO OJEDA

INTEGRANTE

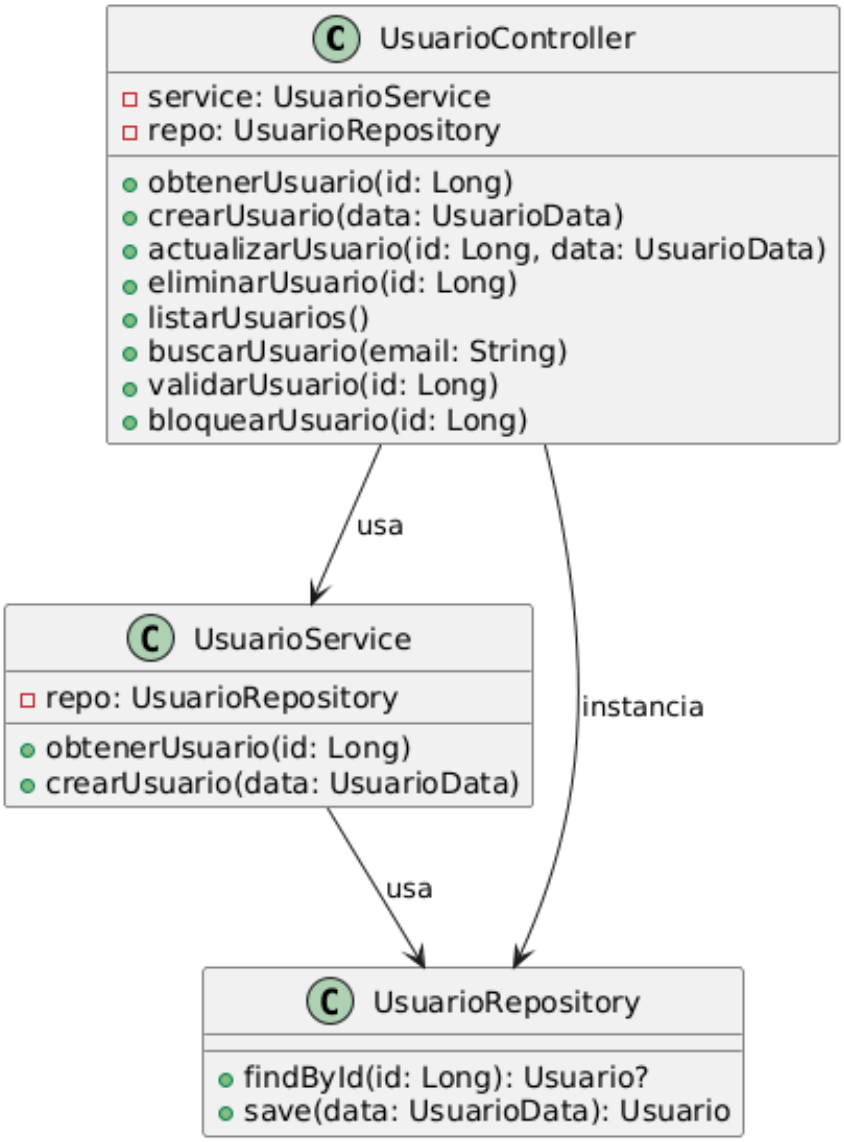
JUAN DAVID RENGIFO CHAVEZ

UNIDADES TECONOLOGICAS DE SANTANDER
PATRONES DE SOFTWARE
INGENIERIA EN SISTEMAS
2026

EJERCICIO DE CLASE (CODIGO)

```
class UsuarioController {
 private val service = UsuarioService()
 private val repo = UsuarioRepository()

 fun obtenerUsuario(id: Long) = service.obtenerUsuario(id)
 fun crearUsuario(data: UsuarioData) = service.crearUsuario(data)
 fun actualizarUsuario(id: Long, data: UsuarioData) =
 service.actualizarUsuario(id, data)
 fun eliminarUsuario(id: Long) = service.eliminarUsuario(id)
 fun listarUsuarios() = service.listarUsuarios()
 fun buscarUsuario(email: String) = service.buscarUsuario(email)
 fun validarUsuario(id: Long) = service.validarUsuario(id)
 fun bloquearUsuario(id: Long) = service.bloquearUsuario(id)
}

class UsuarioService {
 private val repo = UsuarioRepository()

 fun obtenerUsuario(id: Long) = repo.findById(id)
 fun crearUsuario(data: UsuarioData) = repo.save(data)
}

class UsuarioRepository {
 fun findById(id: Long): Usuario? = null
 fun save(data: UsuarioData): Usuario = Usuario()
}
```

1. DIAGRAMA DE CLASES UML



2. ANÁLISIS DE LOS PRINCIPIOS SOLID

2.1 Single Responsibility Principle (SRP)

Definición:

El principio de Responsabilidad Única establece que una clase debe tener una sola razón para cambiar, es decir, debe encargarse de una única responsabilidad dentro del sistema.

No se refiere únicamente a que tenga pocos métodos, sino a que todos sus comportamientos estén alineados con una misma finalidad dentro del dominio.

Cuando una clase asume múltiples responsabilidades:

- Se vuelve más difícil de mantener.
- Aumenta el riesgo de efectos colaterales ante cambios.
- Disminuye la cohesión.
- Incrementa la complejidad estructural.

Violación identificada:

```
class UsuarioController {
    private val service = UsuarioService()
    private val repo = UsuarioRepository()

    fun obtenerUsuario(id: Long) = service.obtenerUsuario(id)
    fun crearUsuario(data: UsuarioData) = service.crearUsuario(data)
    fun actualizarUsuario(id: Long, data: UsuarioData) =
        service.actualizarUsuario(id, data)
    fun eliminarUsuario(id: Long) = service.eliminarUsuario(id)
    fun listarUsuarios() = service.listarUsuarios()
    fun buscarUsuario(email: String) = service.buscarUsuario(email)
    fun validarUsuario(id: Long) = service.validarUsuario(id)
    fun bloquearUsuario(id: Long) = service.bloquearUsuario(id)
}
```

Análisis:

El UsuarioController actualmente:

- Gestiona múltiples operaciones del dominio Usuario.
- Declara una dependencia (UsuarioRepository) que no utiliza.
- Está preparado para crecer agregando nuevas funciones.

Aunque delega la lógica al servicio, su estructura lo expone a múltiples razones de cambio:

- Cambios en endpoints.
- Cambios en reglas de autorización.
- Cambios en validaciones previas a la lógica.
- Cambios en la estructura de datos de entrada o salida.
- Cambios en infraestructura (debido a instanciación directa).

Esto indica que la clase puede evolucionar hacia una clase con múltiples responsabilidades, afectando su cohesión.

El problema no es la cantidad de métodos, sino la concentración progresiva de responsabilidades relacionadas con diferentes aspectos del sistema.

2.2 Open/Closed Principle (OCP)

Definición:

El principio Abierto/Cerrado establece que las entidades de software deben estar:

- Abiertas para extensión.
- Cerradas para modificación.

Esto significa que el comportamiento de una clase debe poder ampliarse sin modificar su código fuente, normalmente mediante abstracciones (interfaces o herencia controlada).

El objetivo es evitar que cada nueva funcionalidad obligue a alterar código ya probado y estable.

Violación identificada:

```
private val service = UsuarioService()
private val repo = UsuarioRepository()
```

Análisis:

El sistema depende directamente de implementaciones concretas.

Si se requiere:

- Cambiar la base de datos.
- Incorporar un repositorio con caché.
- Agregar un repositorio remoto.
- Crear una implementación mock para pruebas.

Se debe modificar el código existente para instanciar la nueva implementación.

Esto significa que el sistema no está cerrado a modificación. Cada cambio estructural implica alterar clases existentes, lo cual aumenta el riesgo de introducir errores.

El diseño no permite extensión por polimorfismo, ya que no existen contratos que permitan intercambiar implementaciones.

2.3 Liskov Substitution Principle (LSP)

Definición:

El principio de Sustitución de Liskov establece que cualquier objeto de una clase derivada debe poder sustituir a su clase base sin alterar la corrección del programa.

En términos prácticos:

- Las implementaciones deben respetar el contrato definido.
- No deben alterar expectativas de comportamiento.
- No deben introducir efectos secundarios inesperados.

Este principio garantiza coherencia y confiabilidad en el uso del polimorfismo.

Violación identificada:

Dependencia directa de clases concretas:

```
private val repo = UsuarioRepository()
```

Análisis:

El diseño actual no utiliza abstracciones ni contratos formales.

Al depender directamente de clases concretas:

- No existe un contrato que defina comportamiento esperado.
- No es posible sustituir implementaciones sin modificar código.
- No se garantiza consistencia en futuras implementaciones.

El sistema está estructuralmente cerrado a la sustitución.

Aunque no hay herencia incorrecta, el diseño impide aplicar LSP correctamente porque no hay mecanismos de sustitución.

2.4 Interface Segregation Principle (ISP)

Definición:

El principio de Segregación de Interfaces establece que ninguna clase debe depender de métodos que no utiliza.

Las interfaces deben ser:

- Específicas.
- Cohesionadas.
- Orientadas al cliente que las consume.

Una interfaz demasiado amplia obliga a las clases a implementar métodos innecesarios, aumentando el acoplamiento y reduciendo claridad.

Violación identificada:

El controlador define múltiples operaciones:

```
fun actualizarUsuario(id: Long, data: UsuarioData)
fun eliminarUsuario(id: Long)
fun listarUsuarios()
fun buscarUsuario(email: String)
fun validarUsuario(id: Long)
fun bloquearUsuario(id: Long)
```

Análisis:

Si se crea una única interfaz que contenga todas estas operaciones:

```
interface UsuarioService {
    // todos los métodos
}
```

Cualquier implementación estaría obligada a implementar funcionalidades que quizá no necesite.

Esto genera:

- Interfaces demasiado amplias.
- Baja cohesión.
- Implementaciones artificiales o vacías.

2.5 Dependency Inversion Principle (DIP)

Definición:

El principio de Inversión de Dependencias establece que:

1. Los módulos de alto nivel no deben depender de módulos de bajo nivel.
2. Ambos deben depender de abstracciones.
3. Las abstracciones no deben depender de detalles.
4. Los detalles deben depender de abstracciones.

Este principio busca reducir el acoplamiento estructural entre capas del sistema.

Violación identificada:

```
private val service = UsuarioService()
private val repo = UsuarioRepository()
```

Análisis:

El UsuarioController (capa de alto nivel) depende directamente de UsuarioService (implementación concreta).

El UsuarioService depende directamente de UsuarioRepository.

Esto provoca:

- Acoplamiento fuerte.
- Rigidez estructural.
- Dificultad para pruebas unitarias.
- Dependencia directa de infraestructura.
- Imposibilidad de intercambiar implementaciones sin modificar código.

Es la violación más clara y estructural en el diseño presentado por el código de ejemplo.

3. RESUMEN DE VIOLACIONES A LOS PRINCIPIOS SOLID

Principio	Sección del Código que Viola	Problema Identificado	Impacto en el Sistema
SRP	UsuarioController con múltiples métodos y dependencia innecesaria del repositorio	Concentración progresiva de responsabilidades y baja cohesión	Dificulta mantenimiento y escalabilidad; riesgo de convertirse en clase monolítica
OCP	private val service =UsuarioService() private val repo = UsuarioRepository()	Dependencia directa de implementaciones concretas	Cada cambio de infraestructura requiere modificar código existente
LSP	Dependencia directa de clases concretas sin contrato formal	No existe posibilidad de sustituir implementaciones sin modificar código	Limita extensibilidad y reemplazo transparente de componentes
ISP	Tendencia a centralizar múltiples operaciones en una sola interfaz de servicio	Posible creación de interfaces amplias y poco cohesionadas	Implementaciones forzadas a depender de métodos que no utilizan
DIP	Instanciación directa en todas las capas	Alto acoplamiento estructural entre capas	Baja flexibilidad, dificultad para pruebas unitarias y mantenimiento